

Cryptanalysis of HiAE

Alexander Bille¹[0009–0007–4117–8154] and
Elmar Tischhauser¹[0000–0002–2031–8652]

Dept. of Mathematics and Computer Science, University of Marburg, Germany
`{bille,tischhauser}@informatik.uni-marburg.de`

Abstract. We describe key recovery attacks on the authenticated stream cipher HiAE, which was recently proposed for future high-throughput communication networks such as 6G by Huawei. HiAE uses a 2048-bit state, a 256-bit key and produces 128-bit tags, targeting 256-bit security against key and state recovery. As a nonce-based AEAD scheme, it relies on the uniqueness of the nonce per key for these security claims. Our analysis indicates that a complete recovery of the 256-bit key of HiAE is possible with a complexity of 2^{128} data and at most $2^{129.585}$ time in the nonce-respecting attack setting, with various small tradeoffs concerning the data and time complexity. While infeasible in practice, this attack therefore violates the 256-bit security claim for HiAE. We describe further complete key-recovery attacks in the nonce-misuse and release of unverified plaintext (RUP) settings which require only a small constant number of repeated nonces or unverified decryption queries, respectively.

1 The HiAE authenticated stream cipher

HiAE [1] is a scheme for authenticated encryption with associated data (AEAD) from Huawei International Pte. Ltd. with a stream cipher structure using the unkeyed AES round function A . It is designed for high throughput for future communication networks (6G) on Intel/AMD and especially ARM microprocessors with AES instructions. The algorithm uses a 2048-bit state divided into 16 128-bit blocks, has a 256-bit key which is split up into two parts K_0 and K_1 , a 128-bit nonce and produces 128-bit message authentication tags (short tag).

At the beginning of the encryption process, the state is initialized with the nonce, the key parts, two constants C_0 and C_1 and XORs of the aforementioned values. The main process of HiAE is visualized in Figure 1 and consists of five steps, namely initialization, key addition, associated data processing, encryption and finalization. The tag is the XOR of all states after the finalization step. We ignore the AD processing step since for our attacks this step is not used.

The update function is shown in Figure 2 and works on a 16×128 -bit non-linear feedback shift register (NFSR), where the new rightmost state block is computed depending on an 128-bit input from the message or associated data.

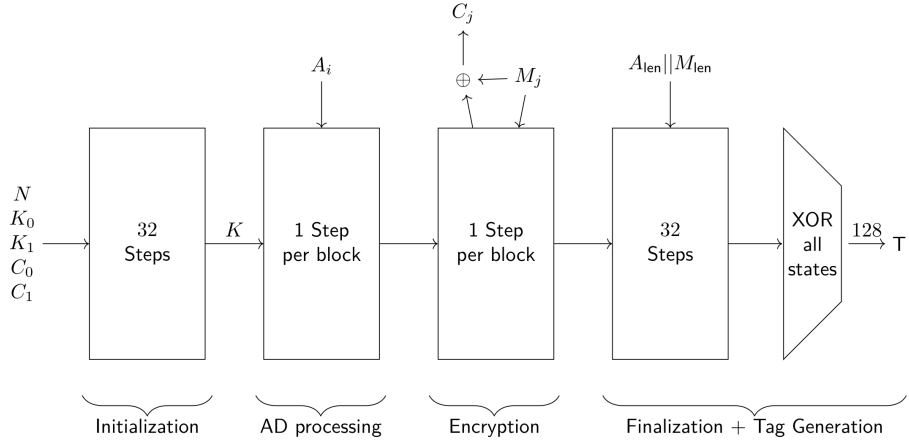


Fig. 1. Encryption process of HiAE (figure from [1]). “Step” means the application of the state update function. C_0 and C_1 in the left side corresponding to the constants C_0 and C_1 respectively.

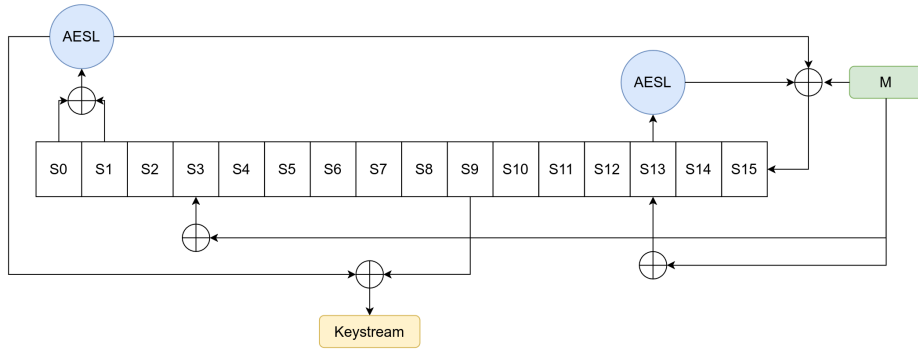


Fig. 2. Update function of HiAE with input M and representation of the state. AESL means the application of the unkeyed AES round function A . (Figure from [1]).

Notation. Let $\parallel$ be the concatenation of two bit strings. Our attacks are in the single key scenario and do not work with different nonces. Hence, throughout this paper, let K_0 and K_1 be the two secret key parts and let N be the arbitrary but fixed nonce. Let O_i^M be the i -th keystream block of HiAE with M as the encrypted message M . The ciphertext $C = C_0 \parallel \dots \parallel C_n$ of a message $M = M_0 \parallel \dots \parallel M_n$ is computed as $C_i = M_i \oplus O_i^M$.

Let S_i with $i \in \{0, \dots, 15\}$ be the i -th state block after the key addition step. Note that for HiAE, the AES round function A consists of the application of SubBytes SB, ShiftRows SR and MixColumns MC of AES, so no key addition. Thus we can compute A^{-1} .

2 Analysis of HiAE

2.1 Differential properties of one round AES

In this section we show how to obtain the input $X = x_0 \parallel \dots \parallel x_{15} \in \mathbb{F}_2^{128}$ of $A(X) = \text{MC}(\text{SR}(\text{SB}(X)))$ by introducing differences and observing the output difference.

Single S-box Let $F : \mathbb{F}_2^8 \times \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8$ be the S-box of AES. With an input difference α and an output difference β , $F(x_i) \oplus F(x_i \oplus \alpha) = \beta$, the input x_i can be either one of 2 or 4 possible values according to the Difference Distribution Table (DDT) of the AES S-box. Let $P_{\alpha, \beta} = \{x \mid F(x) \oplus F(x \oplus \alpha) = \beta\}$ be the possible values with input difference α and output difference β . Since in a row with non-zero difference there are exactly 126 2s and one 4, the average number of possible values for x_i equals $\frac{126 \cdot 2}{256} \cdot 2 + \frac{1 \cdot 4}{256} \cdot 4 = 2.03125$.

Note that $P_{\alpha, \beta}$ equals to $\{p_1, p_1 \oplus \alpha\}$ or $\{p_1, p_1 \oplus \alpha, p_2, p_2 \oplus \alpha\}$. We choose a non-zero input difference γ with $\gamma \notin \{a \oplus b \mid a, b \in P_{\alpha, \beta}\}$. In theory, the intersection of $P_{\alpha, \beta}$ and $P_{\gamma, \beta'}$ can include more than one element and thus there would be multiple options for the unknown plaintext x_i . However, in practice there are input differences where this case does not occur, e.g. with $\alpha = 1$ and $\gamma = 2$ the intersection $P_{\alpha, \beta_1} \cap P_{\gamma, \beta_2} \leq 1$ for all possible $\beta_1, \beta_2 \in \{0, \dots, 255\}$. Choosing such input differences yields a unique value for x_i .

Lemma 1. *Let F be the AES S-box.*

1. *Knowing α and $\beta = F(X) \oplus F(X \oplus \alpha)$ for a fixed unknown value X , this value can either have 2 or 4 possible values. On average it has 2.03125 possible values.*
2. *Knowing α_1, α_2 and its the output differences $\beta_1 = F(X) \oplus F(X \oplus \alpha_1)$, $\beta_2 = F(X) \oplus F(X \oplus \alpha_2)$ the value X has a unique solution.*

AES function A The ShiftRows operation and the MixColumns operation are linear functions.

When dealing with differences the difference through a linear function goes to another difference with probability one. Thus, if we have $\beta = A(X) \oplus A(X \oplus \alpha)$ we can compute it to $\beta' = \text{SB}(X) \oplus \text{SB}(X \oplus \alpha)$ with $\beta' = \text{MC}^{-1}(\text{SR}^{-1}(\beta))$.

The SubBytes operation applies the S-box function F on each byte. The following is deduced by the application of *Lemma 1*:

Lemma 2. *Let $\alpha \neq \alpha' \in \mathbb{F}_2^{128}$ be known and not zero byte wise.*

1. *Knowing the output difference $A(X) \oplus A(X \oplus \alpha)$ we can deduce the fixed unknown input X by at least 2^{16} , in average $2^{16.358}$ and at most 2^{32} guesses.*
2. *Knowing the output differences $A(X) \oplus A(X \oplus \alpha)$ and $A(X) \oplus A(X \oplus \alpha')$ we can deduce the fixed unknown input X exactly.*

2.2 Efficient key recovery from states

We present multiple ways to obtain the key from knowledge of $S_0, \dots, S_{15}$. Note that the complete state is first initialized with the key parts, the constants and the nonce. Afterwards 32 steps of the update function are applied with $\mathcal{C}_0$ as the input. Now consider the formulas for the states $S_0, \dots, S_{15}$ depending on $K_0, K_1, \mathcal{C}_0, \mathcal{C}_1$, and N . For the states starting with S_3 , we observe a reoccurrence of a previous state enveloped in an AES round function as seen in the following:

$$\begin{aligned}
S_3 &= V_3 \oplus A(S_0 \oplus \mathcal{C}_0) \\
S_4 &= V_4 \oplus A(S_1 \oplus \mathcal{C}_0) \\
S_5 &= V_5 \oplus A(S_2 \oplus \mathcal{C}_0) \\
S_6 &= V_6 \oplus A(S_3) \\
S_7 &= V_7 \oplus A(S_4) \\
S_8 &= V_8 \oplus A(S_5) \\
S_9 &= V_9 \oplus A(S_6) \oplus K_0 \\
S_{10} &= V_{10} \oplus A(S_7) \\
S_{11} &= V_{11} \oplus A(S_8) \\
&\dots
\end{aligned}$$

where the remaining quantities V_i depend only on $K_0, K_1, \mathcal{C}_0, \mathcal{C}_1$, and N . Since K_1 occurs only once in V_4 and K_0 occurs once in V_5 as seen in Table 1 we get the respecting key parts by calculating

$$\begin{aligned}
V_4 &= S_4 \oplus A(S_1 \oplus \mathcal{C}_0) \\
V_5 &= S_5 \oplus A(S_2 \oplus \mathcal{C}_0)
\end{aligned}$$

and computing backwards until we obtain K_1 and K_0 respectively. Thus we can get the complete key from the states S_1, S_2, S_4 and S_5 .

The complete key can also be computed from only the two states S_1 and S_4 . After obtaining K_1 with S_1 and S_4 , we use state S_1 to get K_0 . We can compute

Table 1. Occurrences of K_0 and K_1 in the formulas.

	K_0	K_1
S_0	1	5
S_1	1	2
S_2	0	2
V_3	1	1
V_4	0	1
V_5	1	0
V_6	1	2
V_7	2	3

backwards since the key part K_0 occurs only once in S_1 and all other parts are known.

Other options for computing the whole key K_0 and K_1 are also possible. For example there is a more sophisticated way to obtain the both key parts from states S_2 , S_5 , S_8 and S_{11} , but the methods outlined above suffice for our concrete attacks. Altogether we note:

Lemma 3. *By the above calculations, the knowledge of the states $S_0, \dots, S_{15}$ immediately yields both key parts K_0 and K_1 .*

2.3 Obtaining all states from $S_3 \oplus S_4, S_4 \oplus S_5, S_5 \oplus S_6, S_6 \oplus S_7$

We present a way to obtain $S_0, \dots, S_{15}$ with only the knowledge of $S_3 \oplus S_4, S_4 \oplus S_5, S_5 \oplus S_6, S_6 \oplus S_7$ and the encryption of a message M which is at least 15 blocks long. Without loss of generality, and for ease of presentation, we use the all-zero message M^0 for this purpose.

Lemma 4. *With the knowledge of one encrypted message with at least 15 blocks and $S_3 \oplus S_4, S_4 \oplus S_5, S_5 \oplus S_6, S_6 \oplus S_7$ one can obtain the states $S_0, \dots, S_{15}$.*

Proof. Consider the resulting formulas of the keystream blocks for the message M^0 .

$$\begin{aligned}
O_0^{M^0} &= A(S_0 \oplus S_1) \oplus S_9 \\
O_1^{M^0} &= A(S_1 \oplus S_2) \oplus S_{10} \\
O_2^{M^0} &= A(S_2 \oplus S_3) \oplus S_{11} \\
O_3^{M^0} &= A(S_3 \oplus S_4) \oplus S_{12} \\
O_4^{M^0} &= A(S_4 \oplus S_5) \oplus S_{13} \\
O_5^{M^0} &= A(S_5 \oplus S_6) \oplus S_{14} \\
O_6^{M^0} &= A(S_6 \oplus S_7) \oplus S_{15} \\
O_7^{M^0} &= A(S_7 \oplus S_8) \oplus A(S_0 \oplus S_1) \oplus A(S_{13})
\end{aligned}$$

$$\begin{aligned}
O_8^{M^0} &= A(S_8 \oplus S_9) \oplus A(S_1 \oplus S_2) \oplus A(S_{14}) \\
O_9^{M^0} &= A(S_9 \oplus S_{10}) \oplus A(S_2 \oplus S_3) \oplus A(S_{15}) \\
O_{10}^{M^0} &= A(S_{10} \oplus S_{11}) \oplus A(S_3 \oplus S_4) \oplus A(A(S_{13}) \oplus A(S_0 \oplus S_1)) \\
O_{11}^{M^0} &= A(S_{11} \oplus S_{12}) \oplus A(S_4 \oplus S_5) \oplus A(A(S_{14}) \oplus A(S_1 \oplus S_2)) \\
O_{12}^{M^0} &= A(S_{12} \oplus S_{13}) \oplus A(S_5 \oplus S_6) \oplus A(A(S_{15}) \oplus A(S_2 \oplus S_3)) \\
O_{13}^{M^0} &= A(S_{13} \oplus S_{14}) \oplus A(S_6 \oplus S_7) \oplus A(A(A(S_0 \oplus S_1) \oplus A(S_{13})) \oplus A(S_3 \oplus S_4)) \\
O_{14}^{M^0} &= A(S_{14} \oplus S_{15}) \oplus A(S_7 \oplus S_8) \oplus A(A(A(S_{14}) \oplus A(S_1 \oplus S_2)) \oplus A(S_4 \oplus S_5))
\end{aligned}$$

In the following we will determine all necessary parts incrementally until we have all state parts. First we obtain $S_{12}, \dots, S_{15}$ by

$$S_{i+9} = O_i^{M^0} \oplus A(S_i \oplus S_{i+1})$$

for $i \in [3, 6]$. Next, we compute the following:

$$\begin{aligned}
S_0 \oplus S_1 &= A^{-1}(A^{-1}(A^{-1}(O_{13}^{M^0} A(S_{13} \oplus S_{14}) \oplus A(S_6 \oplus S_7)) \oplus A(S_3 \oplus S_4)) \oplus A(S_{13})) \\
S_9 &= O_0^{M^0} \oplus A(S_0 \oplus S_1) \\
S_7 \oplus S_8 &= A^{-1}(O_7^{M^0} \oplus A(S_0 \oplus S_1) \oplus A(S_{13})) \\
S_{10} \oplus S_{11} &= A^{-1}(O_{10}^{M^0} \oplus A(S_3 \oplus S_4) \oplus A(A(S_{13}) \oplus A(S_0 \oplus S_1))) \\
S_1 \oplus S_2 &= A^{-1}(A^{-1}(A^{-1}(O_{14}^{M^0} \oplus A(S_{14} \oplus S_{15}) \oplus A(S_7 \oplus S_8)) \oplus A(S_4 \oplus S_5)) \oplus A(S_{14})) \\
S_{10} &= O_1^{M^0} \oplus A(S_1 \oplus S_2) \\
S_{11} &= (S_{10} \oplus S_{11}) \oplus S_{10} \\
S_2 \oplus S_3 &= A^{-1}(O_2^{M^0} \oplus S_{11}) \\
S_8 \oplus S_9 &= A^{-1}(O_8^{M^0} \oplus A(S_1 \oplus S_2) \oplus A(S_{14})).
\end{aligned}$$

At last, we compute backwards from $i = 8$ down to $i = 0$ to obtain the remaining states.

$$S_i = (S_i \oplus S_{i+1}) \oplus S_i$$

The combination of Lemma 4 and Lemma 3 is stated in Theorem 1.

Theorem 1. *With the knowledge of $S_3 \oplus S_4, S_4 \oplus S_5, S_5 \oplus S_6, S_6 \oplus S_7$ and the first 15 keystream blocks the secret key parts K_0 and K_1 are immediately determined.*

2.4 Model-compliant (nonce-respecting) attacks

In this section, we show how to fulfill the requirements for Theorem 1 by a single encryption (C, T) of a some message M and some decryption queries. These attacks respect the model of unique nonces, e.g. never request two message encryptions with the same nonce and same key. For the cryptanalysis of

Rocca, Hosoyamada et al. [2] introduced the idea to ask for the decryption of (N, C', T') with $C' \neq C$ and a random tag T' to obtain a second message with the same nonce within the nonce-respecting model. We use this principle to set specific message blocks to our requirements which act like differences to obtain the required state values Theorem 1.

The first seven keystream blocks have the following formulas for the message $M = M_0 || M_1 || \dots || M_6$.

$$\begin{aligned} O_0^M &= A(S_0 \oplus S_1) \oplus S_9 \\ O_1^M &= A(S_1 \oplus S_2) \oplus S_{10} \\ O_2^M &= A(S_2 \oplus S_3 \oplus M_0) \oplus S_{11} \\ O_3^M &= A(S_3 \oplus S_4 \oplus M_0 \oplus M_1) \oplus S_{12} \\ O_4^M &= A(S_4 \oplus S_5 \oplus M_1 \oplus M_2) \oplus S_{13} \oplus M_0 \\ O_5^M &= A(S_5 \oplus S_6 \oplus M_2 \oplus M_3) \oplus S_{14} \oplus M_1 \\ O_6^M &= A(S_6 \oplus S_7 \oplus M_3 \oplus M_4) \oplus S_{15} \oplus M_2 \end{aligned}$$

For readability and without loss of generality we fix the to be encrypted message to a message M^0 consisting of 15 zero-blocks.

The following observation gives us difference pairs to obtain $S_i \oplus S_{i+1}$.

Observation 1 Assume that $(N, C^{j,\alpha} = C_0^{j,\alpha} || C_1^{j,\alpha} || \dots || C_6^{j,\alpha}, T^*)$ is valid. By asking for the decryption of $(N, C^{j,\alpha}, T^*)$ with

$$C_i^{j,\alpha} = \begin{cases} O_i^{M^0}, & \text{if } i \neq j \\ O_i^{M^0} \oplus \alpha, & \text{if } i = j \end{cases}$$

for $i \in \{0, \dots, 6\}$ and for a specific $j \in \{0, \dots, 3\}$, we get the plaintexts

$$\begin{aligned} M_j &= \alpha \\ M_{j+2} &= A(S_{j+2} \oplus S_{j+3}) \oplus A(S_{j+2} \oplus S_{j+3} \oplus \alpha) \\ M_{j+3} &= A(S_{j+3} \oplus S_{j+4}) \oplus A(S_{j+3} \oplus S_{j+4} \oplus \alpha), \end{aligned}$$

and for $k < j$ and $k = j + 1$ the plaintext will be $M_k = 0$.

One decryption query has the probability of $\frac{1}{2^{128}}$ to have a valid tag. Therefore we have to ask at most 2^{128} times until we get the plaintext of our decryption request.

Attack 1. By using Observation 1 twice for $C^{1,\alpha}$ and $C^{3,\alpha}$ with α not zero byte wise, we get

$$\begin{aligned} &A(S_3 \oplus S_4) \oplus A(S_3 \oplus S_4 \oplus \alpha) \\ &A(S_4 \oplus S_5) \oplus A(S_4 \oplus S_5 \oplus \alpha) \\ &A(S_5 \oplus S_6) \oplus A(S_5 \oplus S_6 \oplus \alpha) \\ &A(S_6 \oplus S_7) \oplus A(S_6 \oplus S_7 \oplus \alpha) \end{aligned}$$

We need to guess the values with a time complexity of at least 2^{64} , on average $2^{98.147}$ and at most 2^{128} by Lemma 2. To reduce the guessing complexity, we could ask for more decryptions. For example, by asking for the decryption for $C^{1,\alpha'}$ with $\alpha' \neq \alpha$ byte wise we can deduce $S_3 \oplus S_4$ and $S_4 \oplus S_5$ exactly by Lemma 2. Analogously, we can determine $S_5 \oplus S_6$ and $S_6 \oplus S_7$ exactly with an additional decrypted text with $C^{3,\alpha'}$. Each further decrypted text increases the data complexity since we have to request it around 2^{128} times but reduces the number of guesses needed. The resulting tradeoff is illustrated in Table 2.

Table 2. Overview of the attacks on HiAE in the model respecting case.

Attack no.	#enc	#dec ($\times 2^{128}$)	data	guesses			total time		
				min	avg	max	min	avg	max
1	1	2	2^{129}	2^{64}	$2^{65.432}$	2^{128}	2^{129}	2^{129}	$2^{129.585}$
1	1	3	$2^{129.585}$	2^{32}	$2^{32.716}$	2^{64}	$2^{129.585}$	$2^{129.585}$	$2^{129.585}$
1	1	4	2^{130}	0	0	0	2^{130}	2^{130}	2^{130}
2	1	1	2^{128}	2^{64}	$2^{81.497}$	2^{256}	2^{128}	2^{128}	2^{256}
1+2	1	1-2	$2^{128}-2^{129}$	2^{64}	$2^{81.497}$	2^{128}	2^{128}	2^{128}	$2^{129.585}$

Attack 2. In the following we make a finer estimation allowing an attack based on using only one successfully decrypted ciphertext. With the C' from Observation 1 for $C^{1,\alpha}$ with α not zero byte wise we get

$$\begin{aligned}
M_1 &= \alpha \\
M_3 &= A(S_3 \oplus S_4) \oplus A(S_3 \oplus S_4 \oplus \alpha) \\
M_4 &= A(S_4 \oplus S_5) \oplus A(S_4 \oplus S_5 \oplus \alpha)
\end{aligned}$$

Looking at the next plaintexts we observe that

$$\begin{aligned}
M_5 \oplus \alpha &= A(S_5 \oplus S_6) \oplus A(S_5 \oplus S_6 \oplus M_3) \\
M_6 &= A(S_6 \oplus S_7) \oplus A(S_6 \oplus S_7 \oplus M_3 \oplus M_4)
\end{aligned}$$

Assuming that in the average case, the resulting M_3 and $M_3 \oplus M_4$ will be random values, we have the chance of $\frac{1}{256}$ each that a byte is 0. For such a byte we can not use the differential properties for the AES S-box which leads to having to guess this byte with all 256 possible values. In the other 255 cases of a non zero byte Lemma 2 states that the average number of guesses for a non zero input is 2.03125. Thus the number of guesses for a byte would be

$$\frac{255 \cdot 2.03125 + 1 \cdot 256}{256} \approx 3.023.$$

The input difference M_3 for $S_5 \oplus S_6$ is the output difference with input difference α where α is not zero byte wise. Looking at each column during the

MixColumns operation there are four non zero bytes as the input. Since the branch number of this operation is 5 we know that on each column there is at least one non zero byte. Thus M_3 has at most 12 zero bytes. For $M_3 \oplus M_4$ all 16 bytes can be zero at the same time which yield $2.03125^{36} \cdot 3.023^{28} = 2^{81.497}$ guesses in total on average.

In the worst case, we have $4^{32} = 2^{64}$ guesses for $S_3 \oplus S_4$ and $S_4 \oplus S_5$ in total. if M_3 has 12 zero bytes we need to guess $4^4 \cdot 256^{12} = 2^{104}$ values. Furthermore, if $M_3 = M_4$ the input difference $M_3 \oplus M_4 = 0$ leads to having to guess $S_6 \oplus S_7$ entirely with 2^{128} effort. Overall we would need to guess $2^{64} \cdot 2^{104} \cdot 2^{128} > 2^{256}$ times which is more than bruteforcing the key itself. Hence we would need to guess the key if there are too many zero bytes in M_3 and $M_3 \oplus M_4$. An overview is shown in Table 2.

The best option would be to ask for an additional decryption if the guess effort is higher than 2^{128} which leads to a worst case complexity of $2^{129.585}$ with 2 decrypted texts. The number of guesses depends on the number of zero bytes i in M_3 and $M_3 \oplus M_4$ and the number of the remaining bytes j with 4 possible values. Since there are 64 bytes in total we need to guess

$$k(i, j) = 256^i \cdot 4^j \cdot 2^{64-i-j}$$

with $i \in \{0, \dots, 28\}$ as the number of zero difference bytes and $j \in \{0, \dots, 64-i\}$ as the number of bytes with 4 possibles values. Note that one byte equals zero with probability $\frac{1}{256}$ and that the probability of having four possible values is $\frac{4}{256}$. The total probability for all cases where $k(i, j) > 2^{128}$ is very small since

$$p = \sum_{i,j,k(i,j) > 2^{128}} \binom{i}{28} \left(\frac{1}{256}\right)^i \left(\frac{255}{256}\right)^{28-i} \cdot \binom{j}{64-i} \left(\frac{4}{256}\right)^j \left(\frac{252}{256}\right)^{64-i-j} \approx 2^{-51.573}.$$

Because p is very small the number of average guesses does not change significantly

$$h = 2^{81.496}(1 - p) + 2^{65.432}p \approx 2^{81.496}$$

and for the overall complexity

$$2^{128}(1 - p) + 2^{129}p + h \approx 2^{128}.$$

2.5 Further (model-violating) attacks

In this section, we outline additional attacks on HiAE which are violating its security model in some way, either by assuming nonce misuse through encryption queries with repeated nonces, or the release of unverified plaintext for decryption queries with invalid tags.

Release of Unverified Plaintext (RUP). In the scenario of RUP we get the plaintext of a decryption even if the tag is not valid. This reduces the complexity compared to the model respecting case since we do not have to guess the tag correctly. Therefore we can use the same attacks from Section 2.4 which are summarized in Table 3. Note that we can recover the key with only one encryption and 4 decryption requests immediately.

Table 3. Overview of the attacks on HiAE in the model violating case (release of unverified plaintext (RUP) or nonce-misuse (NM)).

Model	#enc	#dec	data	guesses			total time		
				min	avg	max	min	avg	max
RUP	1	1	2	2^{64}	$2^{81.497}$	2^{256}	2^{64}	$2^{81.497}$	2^{256}
RUP	1	2	3	2^{64}	$2^{65.432}$	2^{128}	2^{64}	$2^{65.432}$	2^{128}
RUP	1	3	4	2^{32}	$2^{32.716}$	2^{64}	2^{32}	$2^{32.716}$	2^{64}
RUP	1	4	5	0	0	0	5	5	5
NM	2	0	2	2^{64}	$2^{65.432}$	2^{128}	2^{64}	$2^{65.432}$	2^{128}
NM	3	0	3	0	0	0	3	3	3

Nonce Misuse (NM). In the nonce misuse setting we can request messages with the same nonce multiple times. In the model-respecting attack and in the RUP attacks we introduced differences in the message blocks through decryption requests. This is not necessary here since we can now introduce them straightforwardly through encryption requests. First we encrypt a message with at least 15 blocks. Without loss of generality let this message M^0 consists of zero blocks only. We ask for the encryption of M^0 and obtain its corresponding output keystream which is stated in Section 2.3.

Consider the message $M^\alpha = 0||\alpha||0||\alpha||0||0||0$. We observe that

$$\begin{aligned}
O_3^{M^0} \oplus O_3^{M^\alpha} &= A(S_3 \oplus S_4) \oplus A(S_3 \oplus S_4 \oplus \alpha) \\
O_4^{M^0} \oplus O_4^{M^\alpha} &= A(S_4 \oplus S_5) \oplus A(S_4 \oplus S_5 \oplus \alpha) \\
O_5^{M^0} \oplus O_5^{M^\alpha} &= A(S_5 \oplus S_6) \oplus A(S_5 \oplus S_6 \oplus \alpha) \\
O_6^{M^0} \oplus O_6^{M^\alpha} &= A(S_6 \oplus S_7) \oplus A(S_6 \oplus S_7 \oplus \alpha)
\end{aligned}$$

Thus, we get the difference by an additional encryption request for M^α with some α not zero byte wise. Now we can either guess the values $S_i \oplus S_{i+1}$ or ask for an additional encryption with α' not zero byte wise and distinct to α byte wise. Because of Lemma 2 we can guess one value with at least 2^{16} and at most 2^{32} guesses and in the latter case we can deduce the necessary values exactly. These attacks are summarized in Table 3.

References

1. Han Chen, Tao Huang, Phuong Pham, and Shuang Wu. HiAE: A high-throughput authenticated encryption algorithm for cross-platform efficiency. *Cryptology ePrint Archive*, Paper 2025/377, 2025.
2. Akinori Hosoyamada, Akiko Inoue, Ryoma Ito, Tetsu Iwata, Kazuhiko Minematsu, Ferdinand Sibleyras, and Yosuke Todo. Cryptanalysis of rocca and feasibility of its security claim. *IACR Trans. Symmetric Cryptol.*, 2022(3):123–151, 2022.